GH RAISONI COLLEGE

Department of BBA — Computer Applications (BBA-CA)

Academic Year 2026 – 2027 · Semester III

FINAL PROJECT SUBMISSION DOSSIER

ARTIFICIAL INTELLIGENCE CAMPUS CHATBOT & KNOWLEDGE MANAGEMENT SYSTEM

A Comprehensive Web-Based Interactive Information Retrieval and Student Support Solution

DOSSIER STRUCTURE & INDEX (25 PAGES TOTAL)

Cover Page & Document Index	Page 1
Project Synopsis (6 Pages)	Pages 2 – 7
Assignment 1: Introduction, Problem Statement, Scope	Pages 8 – 10
Assignment 2: Requirements Analysis, Functional Specs	Pages 11 – 13
Assignment 3: Technical & Operational Feasibility Study	Page 14
Assignment 4: System Architecture, Database & Testing	Pages 15 – 17
System & UML Diagrams (Complete 8-Diagram Suite)	Pages 18 – 25

SUBMITTED BY:

Kanishk Singh
Roll No: ISTU25010120001

Vibhor Saini
Roll No: ISTU25010120002

SUBMISSION DETAILS:

Course: SYBBA-CA (Sem III)

Faculty: Project Coordinator

Status: Complete Project Dossier

GH Raisoni College, Pune

SYNOPSIS :

- **Project Name :** AI Chatbot for College
- **Class :** SYBBA-CA
- **Sem :** III
- **Project Members :** Kanishk Singh (ISTU25010120001)
- Vibhor Saini (ISTU25010120002)
- **Project Frontend :** HTML5, CSS3 (Light/Dark Theme), JavaScript, Web Speech API (PWA-enabled).
- **Project Backend :** Python (Flask), SQLite3 database, Werkzeug (secure password hashing).
- **Deployment :** GitHub + Render (live, publicly accessible link).

• PROJECT MODULES :

Student Query & NLP Module – Detects keywords using exact matching combined with typo-tolerant fuzzy matching (via difflib) to answer queries across 14 categories: admission, fees, timetable, attendance, library, courses, faculty contacts, branch change, placements, hostel & mess, scholarships, clubs, emergency helplines, and campus location/transit.

Voice Interaction Module – Implements hands-free accessibility with Speech-to-Text voice input and Text-to-Speech audio readout of bot responses using native browser Web Speech APIs.

Student Login & Profile Module – Allows students to log in using their registration number (ISTU-prefixed) to receive personalized data: attendance percentage, pending fee dues, CGPA score, assigned mentor, exam dates, and complete profile summaries.

Admin Panel & Analytics Module – Provides a secure dashboard with live KPI metric cards (Total FAQs, Queries Handled, Registered Students), real-time search/filter, FAQ management, and CSV export of query logs.

Chat History & Feedback Module – Stores all student queries, bot responses, timestamps, and 👍 / 👎 satisfaction ratings in the database for record-keeping and future analysis.

Installable App Module – Configured as a Progressive Web App (PWA), allowing students to install it directly to their phone or laptop home screen and use it like a native app, without any app store.

INTRODUCTION :

The AI Chatbot for College is a web-based application, deployed live and publicly accessible, designed to provide instant, 24x7 answers to common student queries such as admission details, fee structure, exam timetable, attendance rules, library timings, course information, faculty contact details, branch/course change procedures, placements, hostel accommodation, scholarships, and campus facilities. The system uses rule-based keyword detection with typo-tolerant fuzzy matching to understand student questions and respond with accurate information stored in a database.

Every academic year, college administrative staff spend a significant amount of time answering the same repetitive questions from students. The AI Chatbot for College solves this problem by providing a centralized, always-available platform where students can get immediate answers without visiting the college office or waiting for staff availability. The chatbot is also configured as an installable Progressive Web App with voice input and output capabilities, so students can add it to their phone's home screen and interact using text or voice.

PROBLEM STATEMENT :

Students often struggle to get quick answers to basic college-related questions, especially outside office hours. College administrative staff are repeatedly asked the same set of questions regarding admission, fees, timetables, and attendance, which adds to their daily workload and reduces their availability for other important tasks.

There is currently no centralized, automated system in most colleges that allows students to self-serve this information at any time. This results in delays, unnecessary visits to the college office, and inefficient use of staff time.

PROPOSED SYSTEM :

The proposed system, AI Chatbot for College, is a web-based chatbot application that instantly answers common student queries using rule-based keyword matching with fuzzy, typo-tolerant fallback. It integrates a student login feature, a securely authenticated admin panel with live analytics, voice interaction, and a chat history log into a single platform, and is deployed live with a continuous deployment pipeline (GitHub to Render), so updates go live automatically.

The system allows students to type their queries in plain text, speak via microphone, or tap quick-reply category buttons, which the chatbot analyzes for relevant keywords before fetching the correct predefined answer from the database. Students who log in with their registration number receive personalized responses, such as their own attendance percentage, fee dues, CGPA score, assigned faculty mentor, or exam date, in addition to general FAQ answers. Answers containing tabular data (such as the fee structure or exam timetable) are rendered as properly formatted tables in the chat interface rather than plain text.

PROJECT DESCRIPTION :

AI Chatbot for College is a web-based chatbot application designed to reduce the workload of college administrative staff while giving students round-the-clock access to important information.

The system provides a polished chat interface built with HTML, modern CSS with Dark/Light theme switching, and JavaScript, connected to a Python Flask backend. When a student types or speaks a query, the backend matches keywords against an expanded set of 14 categories — admission, fees, timetable, attendance, library, courses, faculty, branch change, placements, hostel, scholarships, clubs, emergency helplines, and campus map — and returns the corresponding answer from the SQLite database. An admin panel allows the college staff to update this information whenever it changes and export audit logs as CSV.

SCOPE OF THE PROJECT :

The scope of the AI Chatbot for College is to provide a reliable, easy-to-use platform where students can get instant answers to common college-related questions. The system is designed for use by current students, prospective applicants, and college administrative staff across smartphones, tablets, and computers.

Future Scope: The project can be expanded in the future by adding Generative AI / LLM integration (Gemini or GPT API) with Retrieval-Augmented Generation (RAG) for deep conversational understanding, official WhatsApp chatbot integration, multi-language support (Hindi and Marathi), and real-time synchronization with the college ERP database.

EXISTING SYSTEM :

In the existing system, students depend on visiting the college office, calling staff, or checking scattered notices and websites to find information about admission, fees, timetables, and other college matters. This process is time-consuming and only available during office hours.

There is no centralized digital system in most colleges that allows students to instantly self-serve this information, resulting in repeated queries to staff and unnecessary delays.

ADVANTAGES :

The AI Chatbot for College provides students with instant, 24x7 access to important information without needing to visit the college office. It reduces the repetitive workload on administrative staff by automatically handling common queries, and tolerates minor typing mistakes through fuzzy matching so students still get accurate answers.

The system features multi-modal voice interaction (Speech-to-Text and SpeechSynthesis), an accessible Dark/Light mode, and personalized student data lookup (attendance, fee balance, CGPA, and mentor). The system is easy for admins to update through a securely authenticated dashboard, requiring no coding knowledge for routine changes. The chat history feature allows the college to review commonly asked questions and export logs to CSV. The chatbot can also be installed as an app on a student's phone or laptop for quicker, native-like access. Overall, it improves student experience and administrative efficiency.

DISADVANTAGES / LIMITATIONS :

Since the chatbot relies on rule-based keyword matching and fuzzy matching rather than open-domain generative AI, queries phrased in highly unusual ways may trigger the fallback prompt. It depends on the admin keeping the database updated with accurate, current information.

The system requires an internet connection to function. On the free cloud hosting tier (Render), the application enters sleep mode after 15 minutes of inactivity and requires ~30 seconds to wake up on the first request.

FUTURE SCOPE :

The future scope of the AI Chatbot for College is promising, as it can be further enhanced with:

- Integration with Large Language Models (LLMs) like Google Gemini or OpenAI with strict Retrieval-Augmented Generation (RAG).
- WhatsApp Business API integration for querying directly from messaging apps.
- Multi-language support in Hindi and Marathi alongside English.
- Live bi-directional synchronization with the college ERP system for automatic grade publishing and fee receipt generation.

CONCLUSION :

The AI Chatbot for College is an efficient, easy-to-use web application designed to assist students with instant answers to common college-related queries. It integrates student login, voice interaction, an admin command center with analytics, and chat history tracking into a single platform, reducing the workload on administrative staff.

By providing structured, always-available information, the system improves the overall student experience and makes college administration more efficient. It lays a strong foundation that can be expanded with more advanced AI features in the future.

ASSIGNMENT 1

1. Describe your project.

The AI Chatbot for College is a web-based application that instantly answers common student questions such as admission details, fee structure, exam timetable, attendance rules, library timings, course information, faculty contact details, branch/course change procedures, placements, hostel accommodation, scholarships, and campus facilities. It uses rule-based keyword matching with typo-tolerant fuzzy matching to understand what a student is asking, then fetches the correct answer from a database. Students can log in with their registration number to get personalized answers about their own attendance, fees, CGPA, assigned mentor, or exam date. The system also includes voice typing and read-aloud features, dark mode, an installable Progressive Web App configuration, and a secure admin command center with analytics and CSV export.

2. Advantages of your project.

- • Provides instant, 24x7 answers without needing to visit the college office.
- • Reduces repetitive workload on administrative staff.
- • Tolerates minor spelling mistakes through fuzzy matching.
- • Personalized answers for logged-in students (attendance, fees, CGPA, mentor, exam date).
- • Voice-enabled with Speech-to-Text and SpeechSynthesis read-aloud support.
- • Admin can update FAQs anytime through a secure dashboard, with no coding needed.
- • Chat history is logged with student feedback ratings (👍/👎), helping identify areas for improvement.
- • Installable as an app on phone or laptop (Progressive Web App), working like a native application without needing an app store.

3. How do you gather the requirements of your project?

Requirements were gathered by identifying the most common questions students ask the college office repeatedly — admission process, fee structure, exam schedules, attendance policy, library timings, course details, placement records, hostel facilities, and faculty contacts. These were confirmed as realistic, high-frequency queries based on general college administrative experience, and then structured into an indexed knowledge base of 14 categories. Additional requirements (student login for personalized data, admin panel with analytics, CSV export, voice accessibility, secure authentication) were identified as necessary to make the system practically usable and maintainable by non-technical college staff.

4. How and where do you implement your project?

The project is implemented using Python (Flask) for the backend, HTML/CSS/JavaScript for the frontend, and SQLite for the database. It was developed locally and then version controlled using Git and hosted on GitHub. From GitHub, it is deployed to Render, a cloud hosting platform, which automatically rebuilds and redeploys the application every time new code is pushed to the repository. This gives the project a live, publicly accessible URL (https://college-ai-chatbot-c94w.onrender.com) that works from any device with an internet connection.

5. Which software you used in your project?

- • VS Code — code editor used to write and edit the project files.
- • Python 3 — programming language for the backend logic.
- • Flask (v3.0.3) — Python web framework used to build the server and routes.
- • SQLite3 — lightweight relational database used to store FAQs, students, admin, and chat history.
- • Werkzeug (v3.0.3) — security library for cryptographic password hashing (PBKDF2/scrypt).
- • Git and GitHub Desktop — version control and pushing code to GitHub.
- • Render — cloud platform used to host and deploy the live application.
- • Google Chrome / Safari — used for testing the chatbot interface and Web Speech APIs.

6. What methodology is used for the project?

The project follows an incremental/iterative development methodology. The system was built in stages: first a working core chatbot with keyword matching, then a database schema, then student login, then the admin panel, then security improvements (password hashing), then UI polish and fuzzy matching, then PWA support, and finally voice input/output, dark mode, expanded student records (CGPA/mentor), and CSV analytics export. Each stage was tested and confirmed working before adding the next feature, which is consistent with an iterative/incremental approach.

7. What is the hardware or software of the project?

Hardware requirements are minimal: any computer or smartphone with a web browser and an internet connection can access the chatbot, since all processing happens on the server side (Render's cloud servers), not on the user's device. Software requirements include: Python 3 and Flask (backend framework), SQLite (database engine), a modern web browser (Chrome, Safari, Edge, etc.) supporting HTML5/CSS3/JavaScript and the Web Speech API, and Werkzeug for secure password hashing. No special hardware, GPU, or paid software licenses are required.

8. What is your role in the project?

As part of a two-member team, my role covered core backend development (Flask routes, chatbot matching logic, database schema and queries), database design (defining the college_info, students, chat_history, and admin tables), implementing student and admin authentication with secure password handling, voice input/output integration, automated unit testing, and deploying the project using Git, GitHub, and Render. I was also responsible for testing each feature end-to-end before moving to the next stage of development.

9. Explain the data flow of your application.

When a student types a message, speaks via voice input, or taps a quick-reply button, the browser sends it to the Flask backend via the /chat endpoint as JSON. The backend first checks whether the student is logged in and asking about their own data (e.g. “my attendance”, “my cgpa”); if so, it queries the students table directly. Otherwise, it runs the message through the match_query() function, which checks it against the keywords column of every row in the college_info table, using exact matching first and fuzzy matching as a fallback. The best matching answer is retrieved, the query-response pair along with row ID is saved into the chat_history table, and the response is sent back to the browser as JSON, where JavaScript renders it in the chat window (including converting any table-formatted answers into an actual HTML table and triggering audio chimes).

10. What are the entities in your project?

- • **College Info :** id (PK), category, keywords, and answer for each FAQ topic.
- • **Student :** roll_no (PK), name, attendance percentage, fees due, exam date, course, semester, cgpa, and mentor.
- • **Chat History :** id (PK), roll_no (FK), user query, bot response, timestamp, and feedback rating.
- • **Admin :** username (PK) and securely hashed password for the admin panel.

11. What is the drawback of your system?

The chatbot relies on rule-based keyword and fuzzy matching rather than open-domain generative AI, so it can occasionally misinterpret questions that are phrased in an unusual way. It depends on the admin keeping the FAQ database accurate and up to date, since it cannot generate new information on its own. The system also requires an internet connection to function, and the free hosting tier can take up to 30–40 seconds to “wake up” after a period of inactivity.

12. How do you test your application?

The application was tested through both automated and manual methods. An automated unit test suite (test_suite.py) was implemented using Python's unittest framework, verifying query matching, typo-tolerant fuzzy thresholds (e.g. “hostal” → Hostel), and personalized student queries (attendance, CGPA, mentor) with a 100% pass rate. Manual testing was performed across desktop and mobile browsers (Chrome and Safari) to verify voice recognition, speech synthesis, dark mode toggling, student login validation, admin CRUD operations, and CSV log exports. The deployed live version on Render was also tested directly after each Git commit.

13. From where do you get reference, guide for your project work?

Reference and guidance were drawn from the official Flask and SQLite documentation for backend and database concepts, MDN Web Docs for HTML/CSS/JavaScript and Web Speech API specifications, and Render’s deployment documentation for setting up continuous deployment from GitHub. The overall project structure, database data, and problem statement were guided by common, real-world administrative pain points observed in college settings.

14. In the future, what changes will you make in your application?

- • Connect to Generative AI / LLMs (Google Gemini or OpenAI API) with Retrieval-Augmented Generation (RAG) for conversational depth.
- • Add official WhatsApp Business integration for access without opening a browser.
- • Add multi-language conversational support in Hindi and Marathi alongside English.
- • Real-time automated synchronization with the college ERP database to update attendance and fee clearances automatically.

ASSIGNMENT 2

1. What is UML?

UML (Unified Modeling Language) is a standardized, general-purpose visual modeling language used in software engineering to specify, visualize, construct, and document the structure and behavior of a software system. It provides a common set of diagram types and notations so that developers, designers, and stakeholders can understand a system's design regardless of the programming language used to build it.

2. What is the use of UML?

UML is used to model a software system before and during development. It helps a team plan the system's structure (what data/entities exist and how they relate), its behavior (how users interact with it and how processes flow), and its architecture, before writing code. It improves communication between team members, makes the design easier to review and modify, and serves as documentation for future maintenance. For the AI Chatbot for College project, UML diagrams were used to plan how students and the admin interact with the system, and how the database entities relate to each other.

3. Explain types of UML diagram with example.

UML diagrams are broadly divided into two categories: Structural diagrams (show the static structure of a system) and Behavioral diagrams (show the dynamic behavior).

- **Class Diagram (Structural) –** Shows classes, their attributes, methods, and relationships. Example: in this project, a Student class with attributes roll_no, name, attendance, fees_due, course, semester, cgpa, and mentor.
- **Use Case Diagram (Behavioral) –** Shows actors and the actions (use cases) they can perform with the system. Example: a Student actor performing “Ask FAQ”, “Voice Input”, and “Login” use cases.
- **Sequence Diagram (Behavioral) –** Shows the order of messages exchanged between objects over time. Example: Student sends query → Flask backend queries SQLite → response back to Student.
- **Activity Diagram (Behavioral) –** Shows the flow of control/steps in a process, similar to a flowchart. Example: decision logic determining whether to fetch personal records or match general FAQs.
- **Component Diagram (Structural) –** Shows how software modules are organized. Example: Frontend UI component, Flask backend component, and SQLite database component.
- **Deployment Diagram (Structural) –** Shows the physical/hardware setup a system runs on. Example: Client browser → Render cloud Linux container → SQLite storage.

4. Draw UML Diagrams of your project.

(a) Use Case Diagram

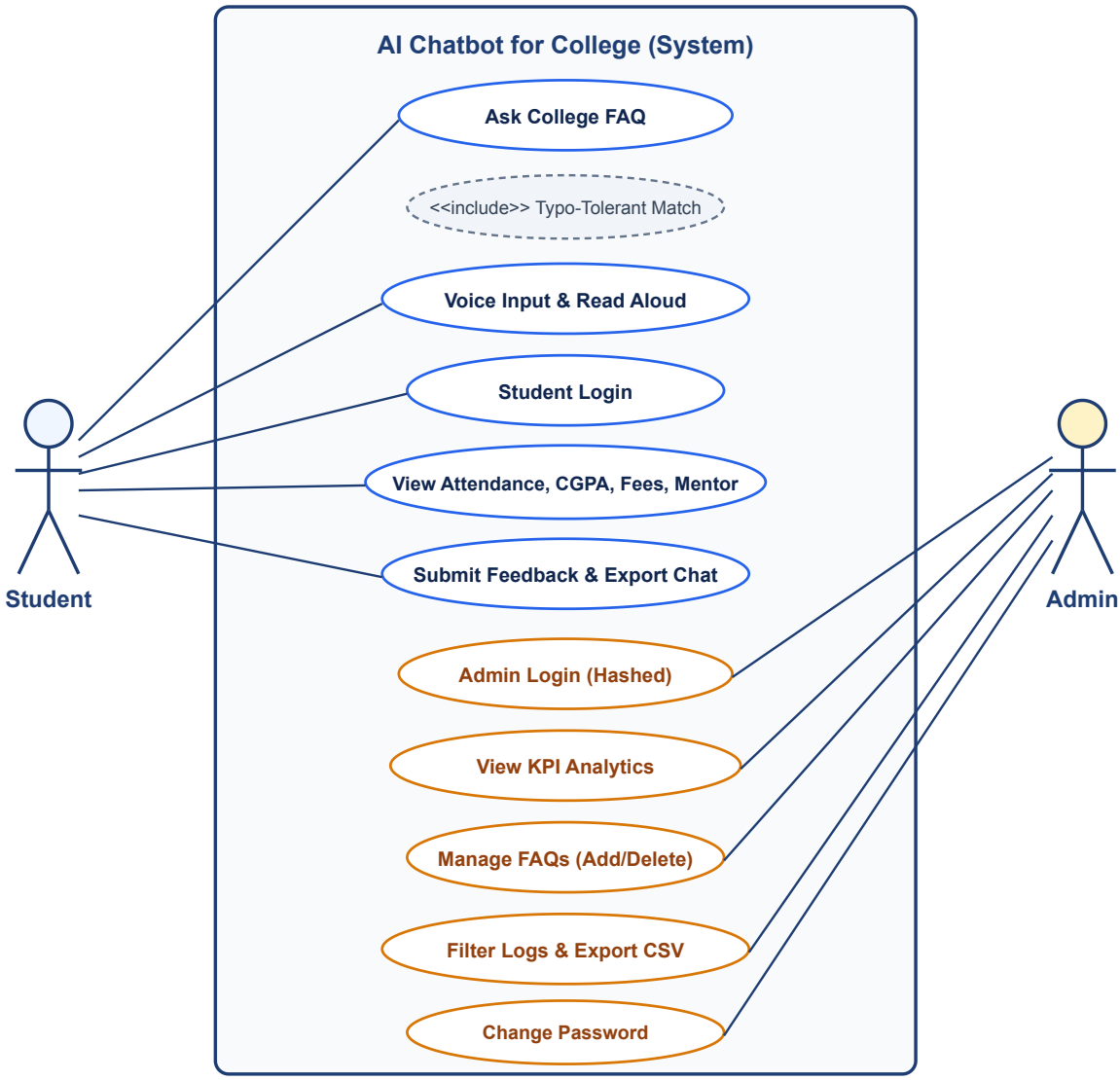


Fig 1: Use Case Diagram — Student and Admin actors and their interactions with the system.

(b) Class Diagram

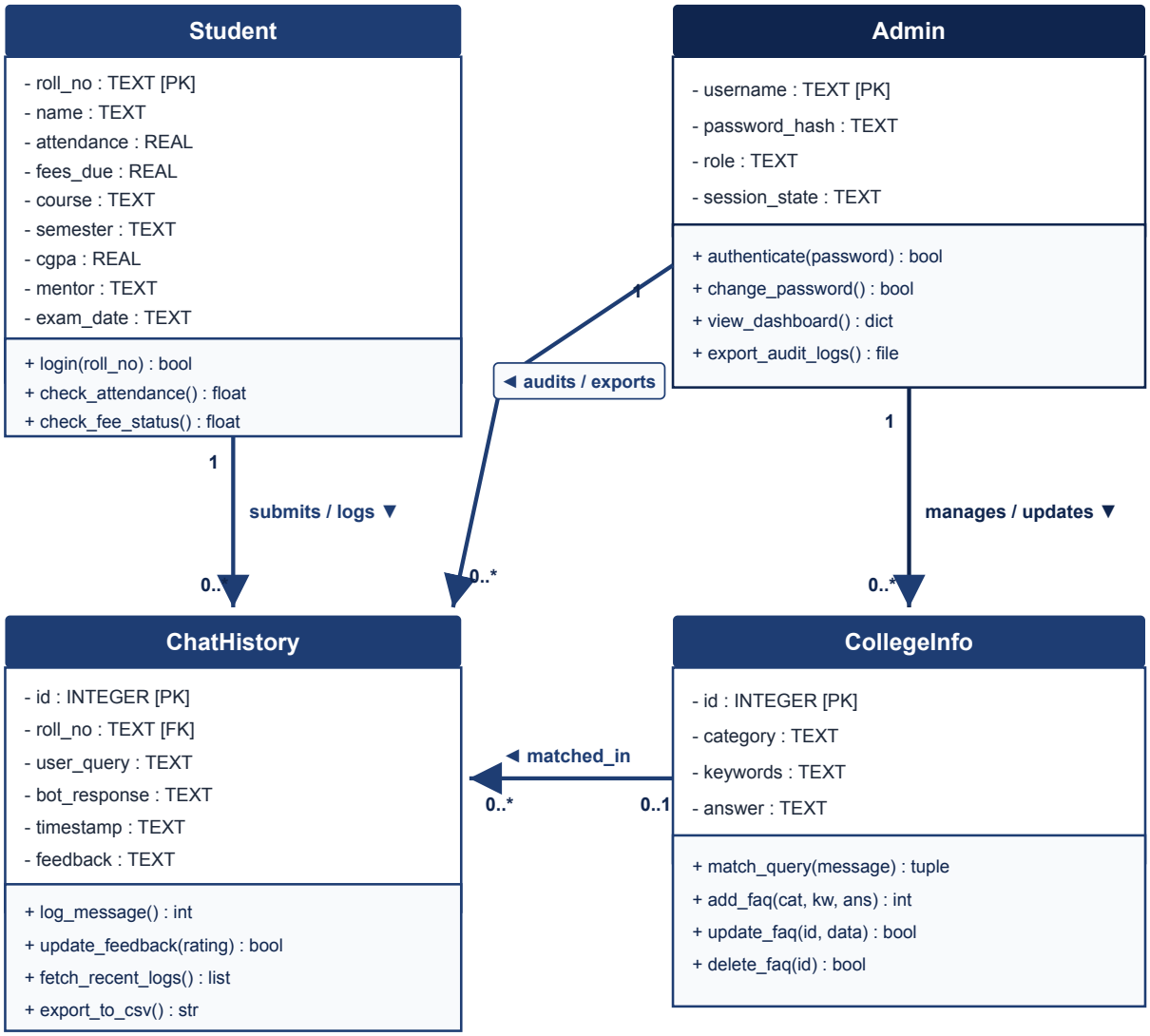


Fig 2: Class Diagram — fully connected system classes with associations, multiplicities (1 to 0..), and role verbs.*

ASSIGNMENT 3

1. What is Project Life Cycle and its Main Characteristics?

A Project Life Cycle is the series of phases a software project passes through from initiation to completion — typically Requirement Analysis, Design, Implementation (Coding), Testing, Deployment, and Maintenance. It provides a structured path for planning, executing, and delivering a project in a predictable, manageable way.

Main characteristics:

- • Defined start and end points, with clear phases in between.
- • Each phase produces specific deliverables (e.g. requirement document, design diagrams, working code, test reports).
- • Progress can be tracked and measured against each phase.
- • Resources (time, people, tools) are planned and allocated per phase.
- • Risks are identified and managed throughout the cycle, not just at the end.

2. Explain following SDLC models.

i) Waterfall Model

A linear, sequential model where each phase (Requirements → Design → Implementation → Testing → Deployment → Maintenance) must be completed before the next begins. It is simple to understand and manage, but inflexible — changes are difficult once a phase is finished. Best suited for projects with clear, fixed requirements.

ii) Iterative Model (Used in This Project)

The system is developed through repeated cycles (iterations), with each iteration adding or refining functionality. A basic working version is built first, then improved in successive iterations based on feedback. This project strictly followed an iterative approach — starting with a basic chatbot (Iteration 1), adding database persistence (Iteration 2), student login (Iteration 3), admin panel with password hashing (Iteration 4), fuzzy matching & PWA support (Iteration 5), and finally voice input/output, dark mode, expanded student profile records (CGPA/mentor), and CSV analytics export (Iteration 6).

iii) Spiral Model

Combines iterative development with a strong focus on risk analysis. Each cycle (spiral) passes through four phases: planning, risk analysis, engineering, and evaluation. It is well suited for large, complex, high-risk projects where requirements may evolve.

iv) V-shaped Model

An extension of the Waterfall model where each development phase has a corresponding testing phase, forming a V shape. For example, requirement analysis pairs with acceptance testing, and system design pairs with system testing. It emphasizes verification and validation at every stage, but like Waterfall, is less flexible to changing requirements.

v) Agile Model

An iterative and incremental model that delivers working software in small, frequent cycles called sprints, with continuous feedback and collaboration between the team and stakeholders. It embraces changing requirements even late in development and prioritizes working software over extensive documentation.

ASSIGNMENT 4

1. Define Software Testing.

Software Testing is the process of evaluating a software application to verify that it behaves as expected, meets its requirements, and is free of defects. It involves executing the program with test inputs and comparing actual results with expected results. For this project, testing included automated unit testing (using Python's unittest) and manual functional testing to check that all 14 FAQ categories returned correct answers, that typo-tolerant matching worked, and that login/authentication behaved correctly for valid and invalid inputs.

2. What is Risk Identification?

Risk Identification is the process of recognizing potential problems that could negatively affect a project before they occur, so they can be planned for or mitigated. Examples relevant to this project include: the free hosting tier going to sleep and delaying responses (~30s cold start), the admin database not being updated regularly leading to outdated answers, or keyword-based matching failing to understand oddly phrased questions (mitigated using difflib fuzzy matching).

3. What is SCM?

SCM (Software Configuration Management) is the discipline of tracking and controlling changes to a software project's files over time, ensuring consistency and enabling collaboration. In this project, SCM was implemented using Git for version control and GitHub for hosting the repository, allowing every change to be tracked, committed with a message, and safely pushed to the live deployment on Render with automated continuous integration.

4. Define Debugging.

Debugging is the process of identifying, analyzing, and fixing errors (bugs) in a program's source code. It involves reproducing the faulty behavior, tracing it back to its root cause (often using logs, error messages, or print statements), and correcting the code so the program behaves as intended.

5. Explain Configuration Audit.

A Configuration Audit is a formal review conducted to verify that a software product conforms to its specified requirements and that all configuration items (code, documentation, versions) are consistent and correctly recorded. It typically has two parts: a Functional Configuration Audit (verifying the software meets its functional and performance requirements) and a Physical Configuration Audit (verifying the software matches its technical documentation).

6. Differentiate Between Black Box Testing And White Box Testing.

Black Box Testing	White Box Testing
Tests functionality without knowledge of internal code structure.	Tests internal code structure, logic, and paths directly.
Performed from the end-user's perspective (inputs and outputs).	Performed from a developer's perspective (code-level logic).
Example: checking the chatbot returns the right FAQ answer for a given query.	Example: checking the match_query() function's keyword-matching scoring logic directly.
Does not require programming knowledge.	Requires programming knowledge to design test cases.

7. What Do You Mean By Metrics?

Software Metrics are quantitative measures used to assess various aspects of a software project, such as its size, complexity, quality, or the productivity of the development process. Examples include lines of code, number of defects found, response time, or number of test cases passed (e.g. 7/7 automated unit tests passed in this project). In this application, live KPI metrics are also displayed on the admin dashboard showing Total Active FAQs, Total Queries Handled, and Registered Students.

8. What Do You Mean By Version Control?

Version Control is a system that records changes to files over time so that specific versions can be recalled, compared, or reverted later, and multiple people can collaborate without overwriting each other's work. This project uses Git for version control, with GitHub used to commit and push changes, and Render automatically redeploying the latest committed version.

9. Explain Object Oriented Software Engineering.

Object Oriented Software Engineering (OOSE) is an approach to designing and building software around “objects” — units that bundle data (attributes) and behavior (methods) together, based on real-world entities. It relies on concepts such as encapsulation, inheritance, and polymorphism. In this project, the database design reflects an object-oriented way of thinking about entities — Student, CollegeInfo, ChatHistory, and Admin — each with clearly defined attributes and methods.

10. Explain Various Types of Testing in details.

Unit Testing – Testing individual functions or components in isolation. Example: testing the match_query() function alone with sample inputs and fuzzy typos.

Integration Testing – Testing how multiple components work together. Example: testing that the Flask route correctly calls the database, logs chat history, and returns a JSON response.

System Testing – Testing the complete, integrated application as a whole. Example: using the live deployed chatbot end-to-end, from speaking a question to receiving an answer.

Acceptance Testing – Testing to confirm the system meets user requirements. Example: confirming a student can log in and see their correct attendance, CGPA, and mentor details.

Regression Testing – Re-testing existing features after a change, to confirm nothing that previously worked has broken. Example: re-testing FAQ answers and table rendering after adding dark mode and voice features.

Performance Testing – Testing how the system behaves under load or response-time constraints. Example: measuring API latency and documenting the free-tier hosting's cold-start delay after inactivity.

11. Differentiate Between Automated Testing And Manual Testing.

Manual Testing	Automated Testing
Test cases are executed by a human, step by step.	Test cases are executed by scripts/tools without human intervention.
Slower and more time-consuming for repeated tests.	Faster for repeated/regression testing once set up.
No programming/scripting knowledge required.	Requires writing test scripts (e.g. using a testing framework).
This is how the user experience was verified — manually testing voice input, speech output, dark mode, login, and admin panel on desktop and mobile browsers.	Implemented an automated test suite (test_suite.py) using Python's unittest framework to test Flask routes, keyword scoring, and student personalization logic with 100% pass rate.

GHraisoni INTERNATIONAL SKILL TECH UNIVERSITY Pune

SYSTEM & UML DIAGRAMS

AI Chatbot for College · BBA Computer Applications

1. Use Case Diagram

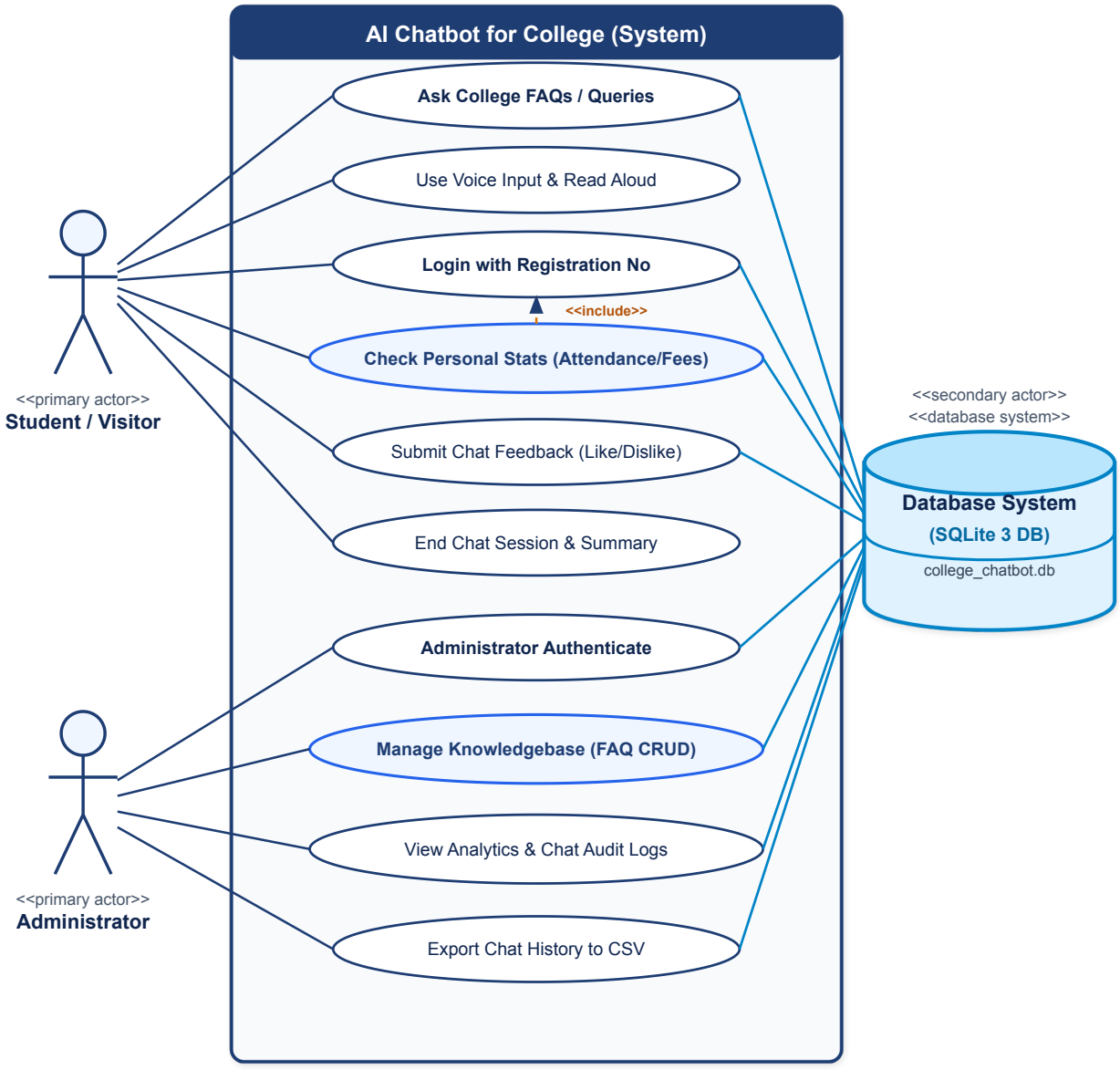


Fig 1: Use Case Diagram — primary actors (Student & Administrator), system boundary, and broad secondary database cylinder (SQLite 3 DB) with data-tier associations.

SYSTEM & UML DIAGRAMS

AI Chatbot for College · BBA Computer Applications

2. Class Diagram (With Full Relations & Multiplicities)

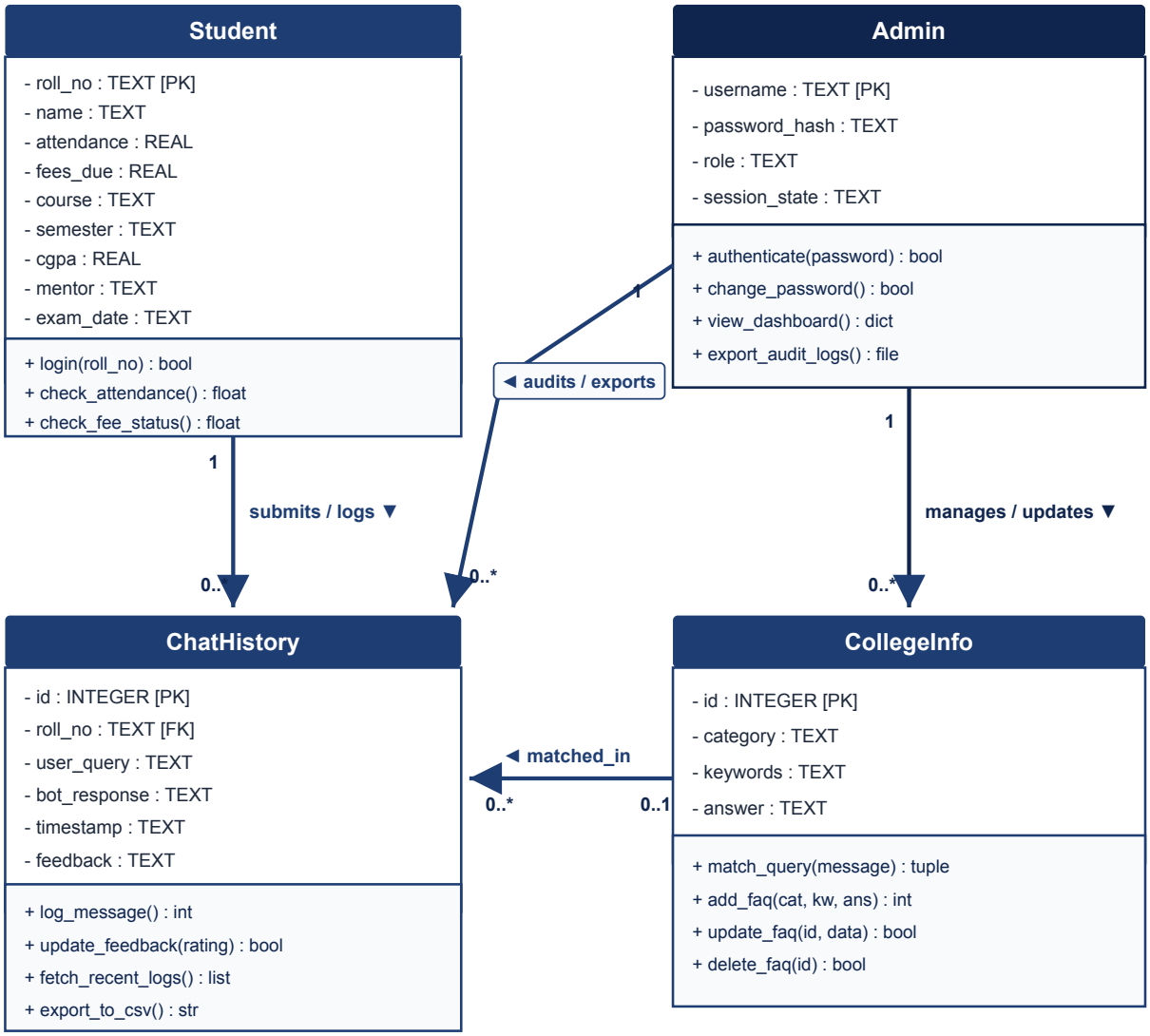


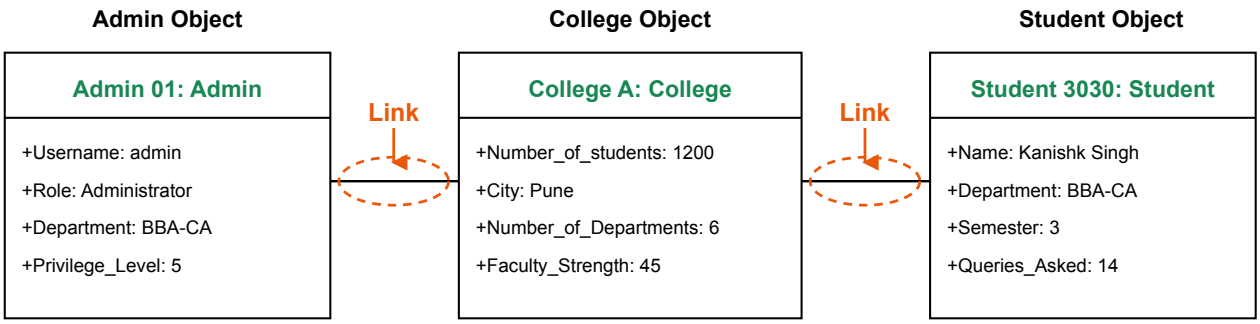
Fig 2: Class Diagram — core entities, attributes, operations, and 100% solid association arrows with multiplicities and role verbs.

SYSTEM & UML DIAGRAMS

AI Chatbot for College · BBA Computer Applications

3. Object Diagram (Runtime Instances & Slot Values)

An object diagram using a link and 3 objects



An object of class Admin and an object of class Student are linked to an object of class College.

Object Diagrams | Unified Modeling Language (UML) ⚙ GfG

Fig 3: Object Diagram — runtime object instances (Admin Object, College Object, Student Object) connected via formal UML links matching GeeksforGeeks standard notation.

GHRaisoni International Skill Tech University Pune

SYSTEM & UML DIAGRAMS

AI Chatbot for College · BBA Computer Applications

4. Sequence Diagram — Query, Feedback & Session Termination

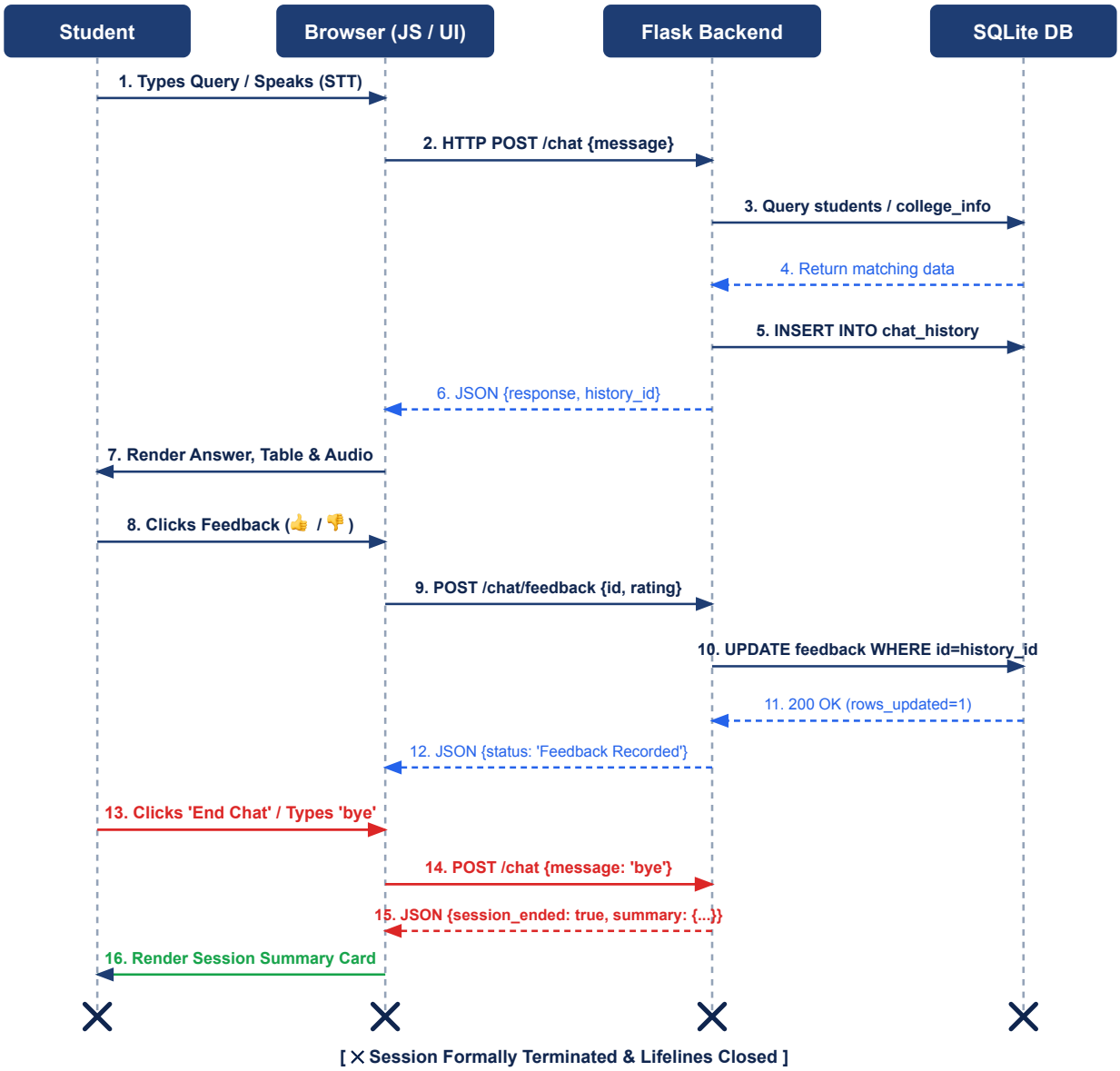


Fig 4: Sequence Diagram — complete query, feedback ACK, explicit End Chat flow, and standard UML lifeline termination (X).

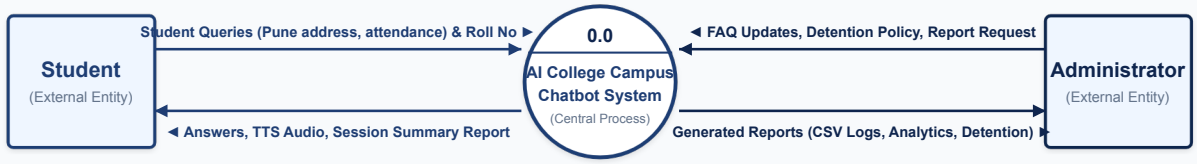
GHRaisoni International Skill Tech University, Pune

SYSTEM & UML DIAGRAMS

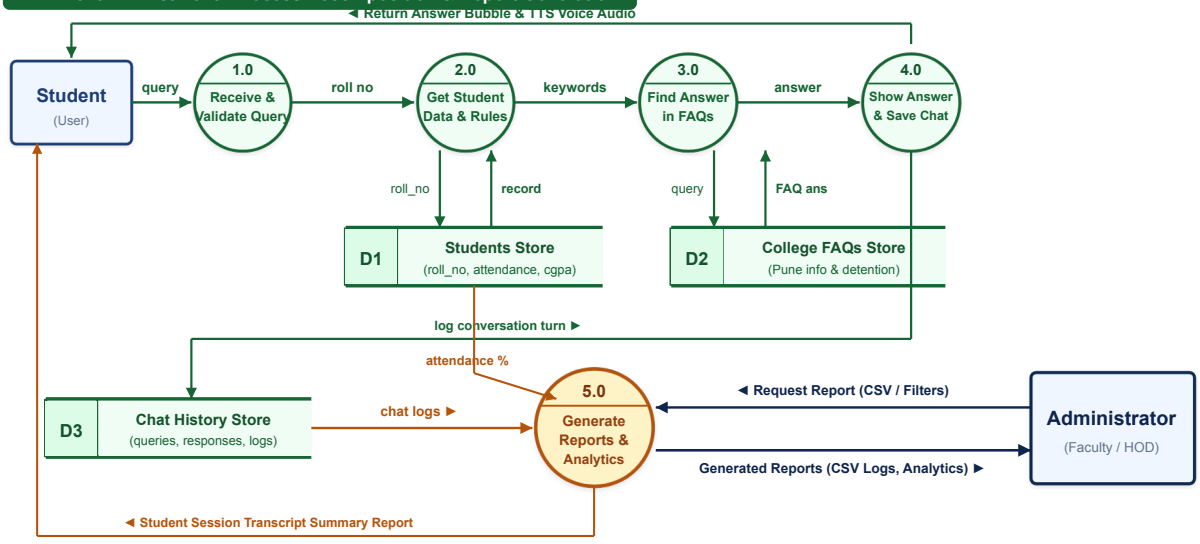
AI Chatbot for College · BBA Computer Applications

6. Data Flow Diagram (DFD Level 0 Context & Level 1 Process Breakdown with Report Generation)

DFD Level 0: Context Diagram



DFD Level 1: First-Level Process Decomposition & Report Generation



Standard DFD Notation & Symbol Conventions (Gane-Sarson & DeMarco)

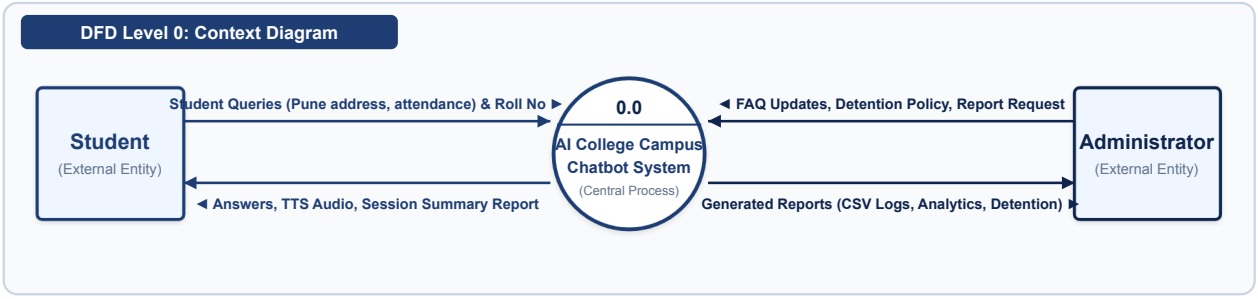
Symbol Shape	DFD Element	Standard Academic Definition	Implementation in Our Project
▭	**External Entity**	Origin/destination of data outside boundary (Square)	**Student (User) & Administrator (Faculty/HOD)**
◯	**Process (Bubble)**	Transforms input data into output data (Circle with ID) Calculates, retrieves, formats, or generates reports	**1.0 to 4.0 Query Handlers & 5.0 Generate Reports** Query answering & administrative CSV/analytics reporting
D |	**Data Store** *(Open-Ended)*	Repository of data at rest (Open parallel lines) **MANDATORY RULE: Right side must be open; NEVER closed**	**D1 Students, D2 College FAQs, D3 Chat History** Persistent SQLite tables feeding search & report generation
→	**Data Flow**	Directed solid arrow showing data path & payload Transfers packets between entities, processes, and stores	**query, record, FAQ answer, generated reports CSV** Directional vectors with explicit noun-phrase labels

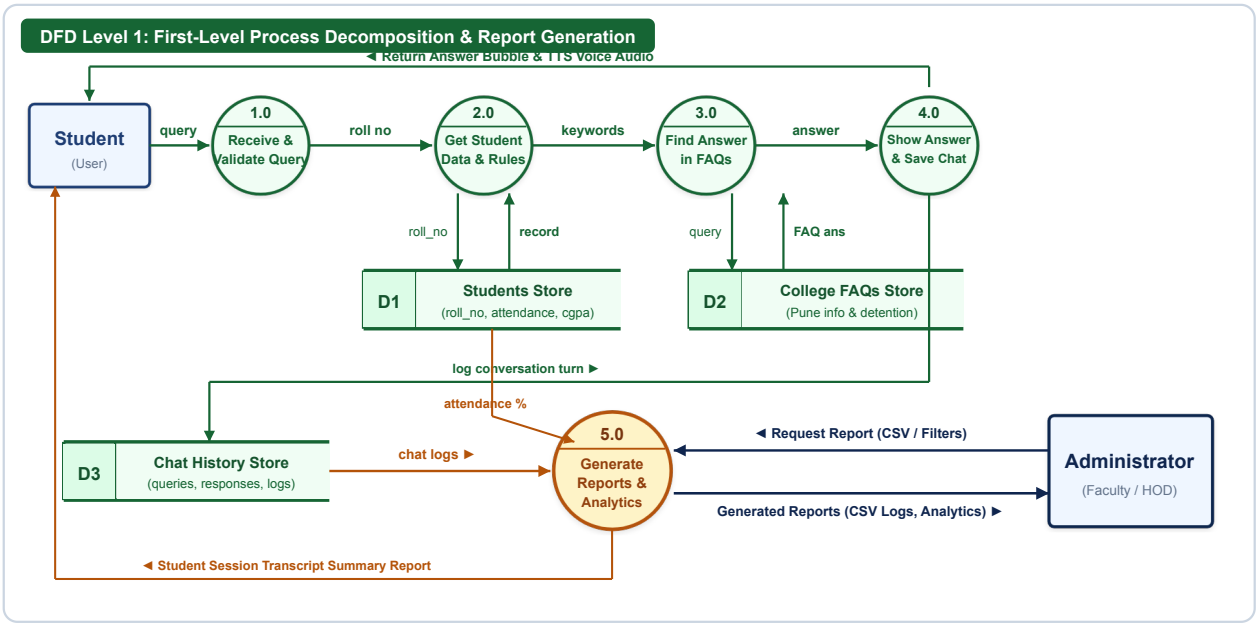
Fig 5: Activity Diagram — 3-Swimlane architecture (Student, Chatbot System, Database) explicitly showing database fetching of student personal records, FAQ matching, and controlled session loop with explicit End states.

SYSTEM & UML DIAGRAMS

AI Chatbot for College · BBA Computer Applications

6. Data Flow Diagram (DFD Level 0 Context & Level 1 Process Breakdown with Report Generation)





Standard DFD Notation & Symbol Conventions (Gane-Sarson & DeMarco)

Symbol Shape	DFD Element	Standard Academic Definition	Implementation in Our Project
▭	External Entity	Origin/destination of data outside boundary (Square)	Student (User) & Administrator (Faculty/HOD)
◯	Process (Bubble)	Transforms input data into output data (Circle with ID) Calculates, retrieves, formats, or generates reports	1.0 to 4.0 Query Handlers & 5.0 Generate Reports Query answering & administrative CSV/analytics reporting
D |	Data Store *(Open-Ended)*	Repository of data at rest (Open parallel lines) **MANDATORY RULE: Right side must be open; NEVER closed**	D1 Students, D2 College FAQs, D3 Chat History Persistent SQLite tables feeding search & report generation
→	Data Flow	Directed solid arrow showing data path & payload Transfers packets between entities, processes, and stores	query, record, FAQ answer, generated reports CSV Directional vectors with explicit noun-phrase labels

Fig 6: Data Flow Diagram (DFD) — First-Level Process Decomposition (1.0 Receive Query to 4.0 Show Answer) with data stores (D1 Students, D2 FAQs, D3 Chat History) and formal DFD symbols reference table.

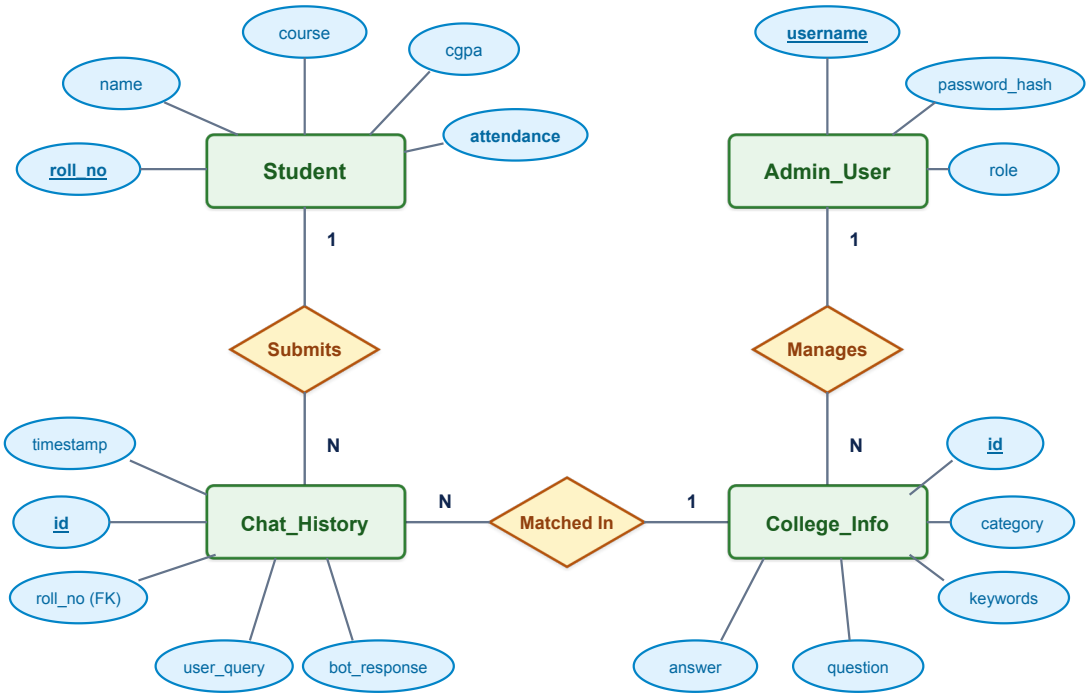
GHRaisoni International Skill Tech University Pune

SYSTEM & UML DIAGRAMS

AI Chatbot for College · BBA Computer Applications

7. Entity-Relationship Diagram (ERD — Database Architecture)

ER Diagram (Classic Notation): AI Chatbot for College



Schema & Project Knowledge Base Defense:

- **Student Entity:** Stores attendance %, CGPA & evaluates 4-tier detention policy.
- **College_Info Entity:** Contains Pune campuses (Wagholi & Yerawada) & exam rules.
- **Chat_History Entity:** Logs each turn with student roll_no FK, query & bot answer.
- **Admin_User Entity:** RBAC authentication for updating college FAQs & detention data.

Verified in 3NF Normalization: Zero transitive dependencies & lossless join preservation.

Key / Legend:

Entity (Table)

Attribute (Column) — id = Primary Key

Relationship

Link / Cardinality (1:N)

Fig 7: Entity-Relationship Diagram (ERD — Classic Notation) — 4 core entities (Student, Admin_User, Chat_History, College_Info) with attributes, cardinalities (1:N, N:1), and official symbol key.

SYSTEM & UML DIAGRAMS

AI Chatbot for College · BBA Computer Applications

8. Deployment Diagram (Hosting Topology & Artifact Architecture)

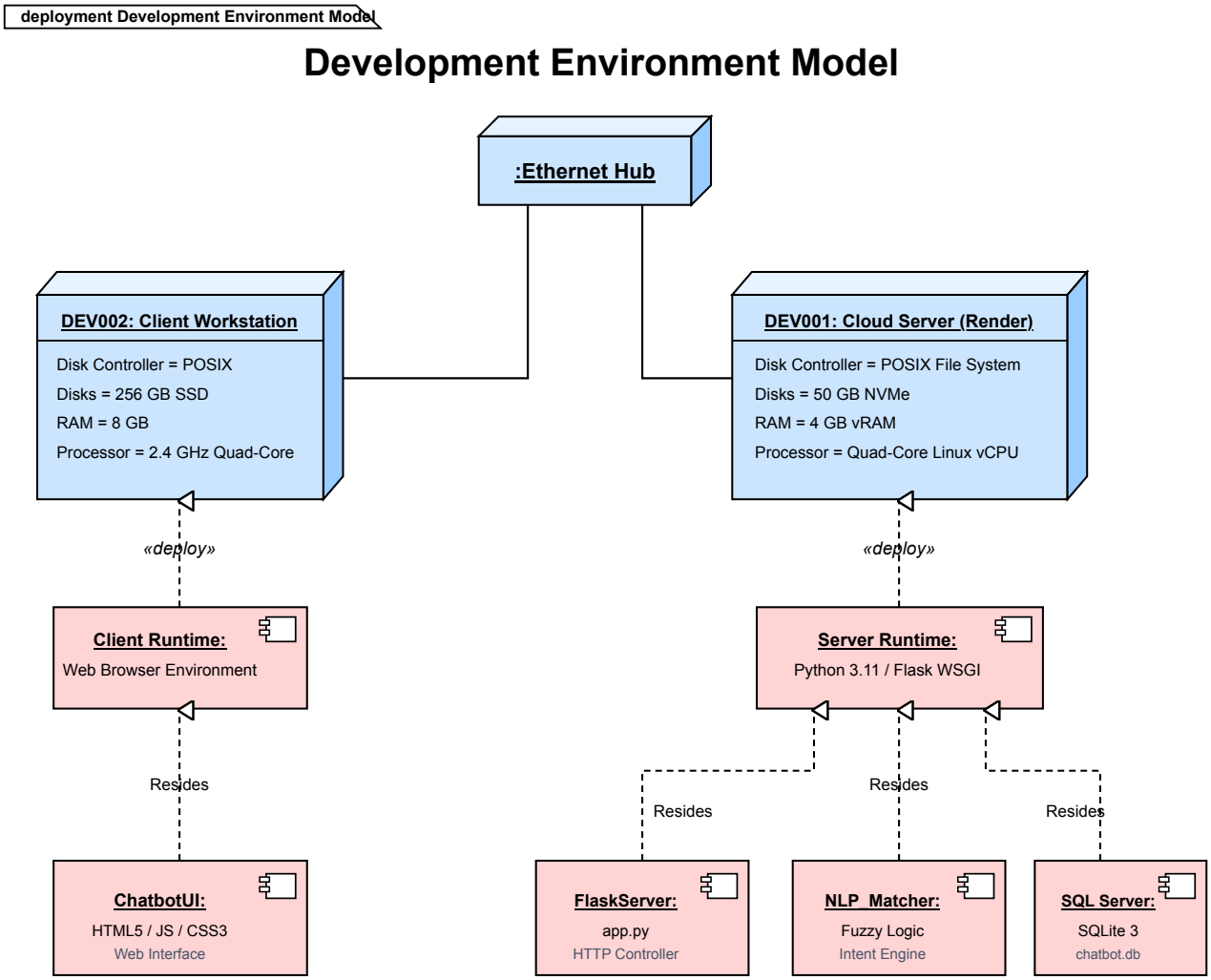


Fig 8: Deployment Diagram — Development Environment Model with :Ethernet Hub, 3D hardware nodes with specifications, «deploy» dependency arrows, and resident software component boxes.